

APPO: Agentic Procedural Policy Optimization

Xucong Wang^{1,2*} Ziyu Ma^{2*} Yong Wang^{2†} Yuxiang Ji² Shidong Yang²
 Guanhua Chen³ Pengkun Wang^{1†} Xiangxiang Chu²
¹USTC, ²AMAP, Alibaba Group, ³SUSTech

Abstract

Recent advances in agentic Reinforcement Learning (RL) have substantially improved the multi-turn tool-use capabilities of large language model agents. However, most existing methods assign credit over coarse heuristic units, such as tool-call boundaries or fixed workflows, making it difficult to identify which intermediate decisions influence downstream outcomes. In this work, we study agentic RL from two perspectives: *where to branch and how to assign credit after branching*. Our pilot analysis shows that influential decision points are broadly distributed throughout the generated sequence rather than concentrated at tool calls, while token entropy alone does not reliably reflect their impact on final outcomes. Motivated by these observations, we propose **Agentic Procedural Policy Optimization (APPO)**, which shifts branching and credit assignment from coarse interaction units to fine-grained decision points in the sequence. APPO selects branching locations using a Branching Score that combines token uncertainty with policy-induced likelihood gains of subsequent continuations, enabling more targeted exploration while filtering out spurious high-entropy positions. It further introduces procedure-level advantage scaling to better distribute credit across branched rollouts. Experiments on 13 benchmarks show that APPO consistently improves strong agentic RL baselines by nearly 4 points, while keeping efficient tool-calls and maintaining behavior interpretability. Project Page: [Github](#).

1 Introduction

Large language models (LLMs) [78, 28, 70, 69, 71, 88, 42] have evolved from static text generators [57] into autonomous agents [74, 94, 10, 13, 48] capable of multi-turn interaction with external environments, enabling strong performance in long-horizon [45, 16] real-world tasks [20, 2, 65, 90]. This progress is largely driven by Reinforcement Learning with Verifiable Rewards (RLVR), which enables policy optimization using sparse outcome-level supervision [18, 60, 34, 58, 61, 5]. However, this training paradigm introduces a fundamental limitation: feedback is only provided at the trajectory level, making it difficult to attribute success or failure to specific intermediate decisions. As a result, each trajectory provides only a coarse and entangled learning signal, leading to inefficient credit assignment and unstable policy improvement [56, 14, 23, 29].

To address this limitation, existing approaches restructure agent rollouts to extract more informative credit signals under limited rollout budgets. A common strategy is to expand trajectories from intermediate locations, constructing multiple candidate branches and assigning credit according to outcome differences across rollouts [91, 101, 29, 98, 62]. The intuition is that, instead of repeatedly sampling full trajectories with substantial redundancy, one can branch around uncertain positions and compare alternative continuations to identify which local decisions lead to better final outcomes. Depending on the branching granularity, this line of work includes workflow-level exploration and tool-call-level branching [23, 10, 8, 38].

*Equal Contribution. Work done during Xucong’s internship at AMAP, Alibaba Group.

†Project lead: Yong Wang; Corresponding authors: Yong Wang and Pengkun Wang

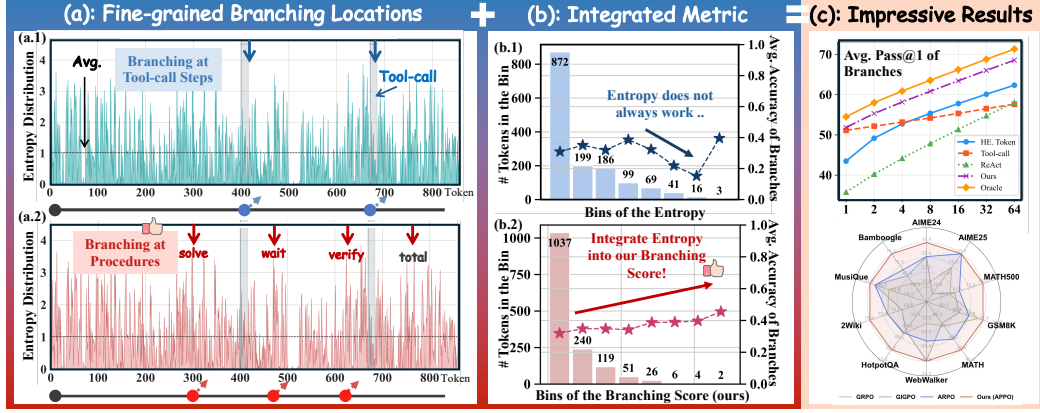


Figure 1: **(a):** The token entropy distribution in the tool-integrated rollout (sampled from Tool-Star’s [9] 54K dataset). **(b):** Average accuracy of branches generated from each token, shown by bins of the entropy and the APPO’s Branching Score (BS). **(c):** The pass@ k of rollouts resampled via different criteria (“oracle” means to resample from the points with the highest accuracy uncertainty); The performance comparison between APPO and others on 10 datasets.

While such designs improve rollout efficiency and partially densify supervision from sparse outcome rewards, they still rely on coarse-grained units for credit assignment. In particular, they often compress the entire non-tool-call process into `<thinking>` tags [10, 8] or rely on fixed workflows [91, 75, 64], thereby overlooking the procedural knowledge embedded within the thinking content [19, 77, 80]. In practice, successful long-horizon agent reasoning is often shaped not by entire thinking blocks or workflow stages, but by a small number of critical *decision points* where alternative continuations lead to substantially different downstream outcomes [77]. These decision points are latent positions in the trajectory, instantiated as tokens in the generated sequence, and identified by their role in inducing divergence in subsequent reasoning paths. We use **procedures** to denote the procedural reasoning patterns organized around such high-impact decision points. Although some individual procedures, such as `plan` [75] and `reflect` [64], have drawn attention in training-free prompt engineering [30, 76], their collective role in online agentic RL remains underexplored.

To further investigate how these procedures relate to reasoning accuracy and failure, we conduct a pilot study summarized in Figure 1, where we analyze **(a)** branching locations and **(b)** the average accuracy of resampled branches under different branching criteria. The results reveal that neither coarse tool-call boundaries nor raw token entropy provides a satisfactory basis for credit assignment. Specifically, Figure 1(a) shows that the highest-uncertainty positions are not concentrated at tool-call boundaries, but are broadly distributed throughout the thinking span, suggesting that non-tool-call reasoning contains finer-grained procedural information beyond coarse tool-call units. Moreover, Figure 1(b.1) shows that high token entropy alone does not reliably indicate decision significance, as tokens with higher entropy do not consistently yield branches with greater outcome uncertainty, implying that some entropy peaks reflect lexical rarity rather than significance to the task outcome.

These findings highlight the necessity of attending to procedural information within the thinking process beyond tool calls, while underscoring the need for a more principled design of branching criteria to enable finer-grained credit assignment across procedures.

Motivated by these findings, we propose Agentic Procedural Policy Optimization (APPO), an agentic RL algorithm that redefines the unit of credit assignment from coarse-grained heuristic units to fine-grained procedures. Given an initial rollout, APPO extends branching-point selection from tool-call boundaries to the entire sequence, and chooses branching tokens using a comprehensive Branching Score (BS). Beyond token entropy, BS measures how much the current policy increases the likelihood of subsequent continuations relative to the old policy, thereby capturing the future value carried by the current token and filtering out spurious high-entropy positions. Building on this design, we further introduce a procedure-level advantage scaling term based on Ω to encourage exploration over procedures with high branching value. We evaluate APPO on 13 challenging benchmarks spanning deep information seeking, knowledge-intensive reasoning, and computational problem

solving, and show that it consistently improves both task performance and exploration flexibility over strong baselines (Figure 1(c)). Our contributions are threefold:

- Our preliminary study demonstrates the critical role of *procedures* in agent reasoning. By moving rollout branching from workflows or tool-call boundaries to procedures, it exposes finer-grained structure within the thinking process and enables more informative intermediate supervision.
- Motivated by these findings, we propose APPO, an agentic RL algorithm that shifts credit assignment from coarse-grained heuristic units to fine-grained procedures. Its Branching Score combines token entropy with the policy-induced likelihood gain of subsequent continuations, enabling the selection of high-value procedures for branching and procedure-level advantage scaling.
- Extensive experiments validate the effectiveness of our method, which outperforms existing approaches by approximately 3 points across 13 benchmarks, while achieving comparable tool-calls and maintaining interpretability.

2 Related Work

Agentic Reinforcement Learning. Reinforcement Learning [32, 60, 58, 18] has proven to be effective in endowing agents with long-horizon complex reasoning and acting capabilities. Beginning from the actor-critic-based PPO [60], a wide range of studies aim to devise more efficient policy variants, or refine the scale of policy gradients, advantages or regularization terms, like GRPO [18], DAPO [92], GSPO [99], GPG [5], Dr. GRPO [44]. More recent research [14, 93, 46] focus on the distinct applications of RL in agent tasks, such as fine-grained credit assignment for actions [14, 10, 8], agent self-evolution [83, 93, 85] and internalization of agent skills [46, 95].

Tree-based RL. Tree-based RL can be roughly divided into the following categories: (i) Offline Training [15, 33, 36, 86], where branches split from the same point are stored as offline preference data and later utilized in DPO [58]. For example, MCTS-DPO [86] compares the estimated reward of candidate nodes and separates them into chosen / rejected samples; SPORT [36] leverages step-wise sampling and automatic verification to formulate step-wise preference data for agent training. (ii) Online Training [23, 29, 10, 8, 98], where the branches are extended from rollouts in each step and used to calculate group-relative advantages. For example, Tree-GRPO [29] randomly selects think-action steps for branching, while ARPO [10] identifies the high-entropy tokens following each tool-call and conducts resampling accordingly. (iii) Test-Time Scaling [76, 66, 84, 79, 52], where the model generates multiple branches to elevate the average performance in inference. Notably, the design of branching location is a critical factor in elevating the performance ceiling of tree-based RL. *In contrast with these lines of studies, APPO treats procedures as finer units of rollout branching in on-policy agentic RL and elicits the reasoning process accordingly.*

3 APPO: Agentic Procedural Policy Optimization

3.1 Preliminaries

Agentic Reinforcement Learning. We consider a standard agentic RL setting [17, 41, 82, 9, 67, 55], where an agent receives a task $x \in \mathcal{X}$ and interacts with a toolset T . The rollout consists of T_a interleaved thinking and tool-call steps, followed by T_b answer generation steps:

$$P_\theta(\mathcal{G}, y|x, T) = \prod_{t=1}^{T_a} [\pi_\theta(\mathcal{O}_t|\mathcal{G}_{<t}, x; T) P_{env}(\mathcal{G}_t|\mathcal{O}_t, \mathcal{G}_{<t})] \prod_{t=1}^{T_b} \pi_\theta(\mathcal{O}_t|y_{<t}, \mathcal{G}, x; T) \quad (1)$$

where P_{env} denotes the external environment. If the agent does not call the external tool-use actions in its outputs $\mathcal{O}_t$, then $\mathcal{O}_t$ equals the overall output $\mathcal{G}_t$ of the step t . Based on the rollouts, the training objective of Agentic Reinforcement Learning process can be formulated as follows:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{X}, y \sim \pi_\theta(\cdot|x; T)} [r_\phi(x, y)] - \beta \mathbb{D}_{KL}[\pi_\theta(y|x; T) || \pi_{ref}(y|x; T)] \quad (2)$$

where π_θ and π_{ref} are the policy and reference models, respectively. r_ϕ is the reward function, and $\mathbb{D}_{KL}$ denotes KL divergence. In practice, rewards are converted into advantages $\hat{A}(\cdot)$ through group-level [61, 18, 44], sequence-level [99], token-level [92], or value-function-based estimators [60].

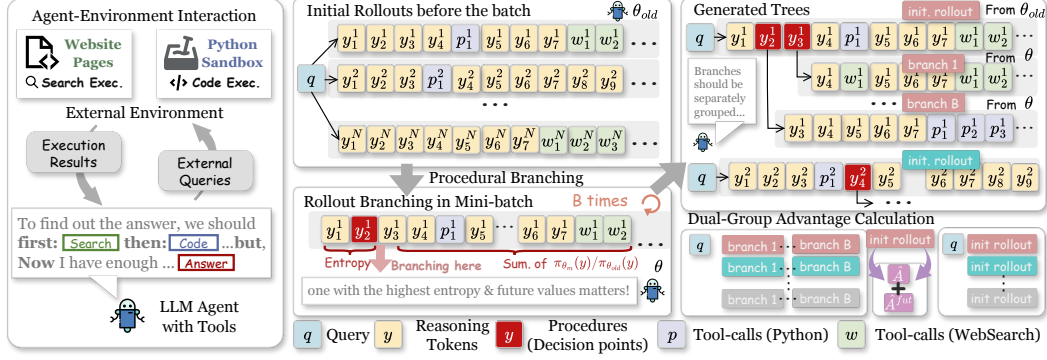


Figure 2: Overview of APPO. The agent first interacts with the environment to generate initial rollouts for each batch. During mini-batch training, APPO identifies fine-grained decision points using the Branching Score (BS), which combines token entropy with future-aware likelihood gains, and resamples continuations from these positions rather than fixed tool-call boundaries. The resulting branches and initial rollouts are then used for dual-group advantage estimation, together with a future-aware advantage term for procedure-level credit assignment.

Token Entropy. A common branching strategy [10, 8, 23] selects tokens with the top- k entropy:

$$H_t = - \sum_{j=1}^{|\mathcal{V}|} p_{t,j} \log p_{t,j}, \quad \mathbf{p}_t = \pi_\theta(\cdot | \mathcal{G}_{<t}, z; T) = \text{Softmax}(\mathbf{z}_t / \tau) \quad (3)$$

where $|\mathcal{V}|$ is the vocabulary size, τ is the temperature, and $\mathbf{z}_t$ are logits. Although entropy reflects local uncertainty, it does not indicate whether a token corresponds to a decision point that changes downstream reasoning. Thus, high-entropy tokens may arise from lexical uncertainty rather than task-relevant procedural choices. APPO addresses this by identifying tokens that instantiate latent decision points, i.e., positions where alternative continuations induce divergent reasoning paths, and uses them to construct procedures for more targeted branching and credit assignment.

3.2 Procedural Rollout Branching

In this section, we introduce the APPO algorithm in detail, designed to guide LLMs in exploring fine-grained procedural-level behaviors under synthetic branching criteria, as illustrated in Figure 2.

► **Initialization.** Given input x and a global rollout budget M , the model first generates N full rollouts $\mathcal{T} = \{\mathcal{H}_n | \mathcal{H}_n \sim \pi_\theta(\cdot | x)\}^N$, which serve as the roots of N independent trees.

► **Sampling.** For each token $\mathcal{H}_{n,i}$ in rollout $\mathcal{H}_n$, APPO measures its future-aware influence using the accumulated decayed importance sampling ratio (π_{old} here is the π_θ in initialization):

$$\Omega_{n,i} = \exp \left(\sum_{i' \geq i} \gamma^{i'-i} \log \rho_{i'}(\theta) \right), \quad \rho_{i'}(\theta) = \frac{\pi_\theta(\mathcal{H}_{n,i'} | \mathcal{H}_{n,<i'}, x; T)}{\pi_{\text{old}}(\mathcal{H}_{n,i'} | \mathcal{H}_{n,<i'}, x; T)}, \quad (4)$$

where π_{old} denotes the policy used to generate the initial rollouts. We name $\Omega_{n,i}$ as the future value. A larger $\Omega_{n,i}$ indicates that the model’s continuations are assigned states that are more favored by training, and vice versa. In this case, we treat $\Omega_{n,i}$ as a replacement of the posterior accuracy-uncertainty, which can only be calculated via innumerable times of Monte-Carlo estimation. Also, to mitigate gradient and variance fluctuations caused by accumulation, we introduce a discount factor γ to suppress the influence of tokens farther away from the current token, thereby reducing variance. We then combine the future-value term with token entropy to define the Branching Score:

$$\text{BS}_{n,i} = \mathcal{Z}(\text{clip}(\Omega_{n,i}, 1 - \epsilon', 1 + \epsilon'); \mathcal{H}_n) \cdot \mathcal{Z}(H_{n,i}; \mathcal{H}_n), \quad (5)$$

where $\mathcal{Z}(\cdot; \mathcal{H}_n)$ denotes z-score normalization within rollout $\mathcal{H}_n$. Entropy captures local uncertainty, while $\Omega_{n,i}$ captures future influence; their product selects tokens that are both uncertain and consequential, serving as a sufficient integration of the sequence prior and posterior.

For each rollout $\mathcal{H}_n$, we select the top- B tokens according to $\text{BS}_{n,i}$ and denote them as $\mathcal{B}_n$ ($|\mathcal{B}_n| = B$). These tokens instantiate latent decision points. We then resample continuations from each $\mathcal{H}_{n,i} \in \mathcal{B}_n$ to generate new branches $\mathcal{H}_n^{\text{new}}$, and update the rollout tree as $\mathcal{H}_n \leftarrow \mathcal{H}_n \cup \mathcal{H}_n^{\text{new}}$.

► **Termination.** Tree expansion stops when either the remaining budget $M - N$ is exhausted or no further branching is performed. It is worth noting that we sample uniformly across all trees. Assume the total number of generation loops is L , then for $L = 1$ (i.e., the source of branching is restricted to the initial rollouts), there is $(B + 1) \cdot N = M$. With a fixed total budget, increasing N allows us to obtain rollouts with a more diverse initial distribution; conversely, increasing B and L enables more branching on a single tree, thereby providing finer-grained credit assignment. We will conduct detailed ablation studies of these coefficients in subsequent sections and Appendix E.

3.3 Procedural Advantage Estimation and Policy Optimization

Most policy optimization methods [29, 87] rely on group-level advantages, which provide limited credit assignment at intermediate decision points. APPO instead assigns credit to procedure-level decisions by using branches as auxiliary contrastive signals. Unlike most Tree RL methods, our branches are generated by the current mini-batch policy π_θ rather than π_{old} . Since gradients are not propagated through all branches, we use them only to compute rewards and advantages.

To avoid bias from mixing rollouts generated by different policies, APPO computes group-relative advantages separately for initial rollouts $\mathcal{T}_{\text{init}}$ and branches $\mathcal{T}_{\text{branch}}$:

$$\hat{A}_{n,i}^{\text{base}} = \text{avg} \left\{ \frac{R_n - \text{mean}(\{R_{n'} \mid \mathcal{H}_{n'} \in \mathcal{T}_*\})}{\text{std}(\{R_{n'} \mid \mathcal{H}_{n'} \in \mathcal{T}_*\})} \mid \mathcal{T}_* \in \{\mathcal{T}_{\text{init}}, \mathcal{T}_{\text{branch}}\} \right\} \quad (6)$$

where R_n is the task reward. Since generated tokens serve as observable instantiations of latent decision points, token-level advantages can be viewed as localized credit assigned to the corresponding procedural decisions. Furthermore, inspired by recent studies [25, 50], we argue that critical procedures offer more “sparse and critical” subsequences; these procedures serve as the turning point of the rollout and more likely to induce higher policy-induced differences over continuations. We then take the similar design of Ω to formulate an extra advantage term, which further emphasizes decisions with stronger downstream influence by assigning larger credits:

$$\hat{A}_{n,i}^{\text{fut}} = \text{clip}_{\epsilon'} \left(\exp \left(\sum_{i' \geq i} \gamma^{i'-i} \log \rho_{i'}(\theta) \right) \right), \quad \rho_{i'}(\theta) = \frac{\pi_\theta(\mathcal{H}_{n,i'} \mid \mathcal{H}_{n,<i'}, x; T)}{\pi_{\text{old}}(\mathcal{H}_{n,i'} \mid \mathcal{H}_{n,<i'}, x; T)} \quad (7)$$

where $\text{clip}_{\epsilon'}$ clips the value into $[1 - \epsilon', 1 + \epsilon']$. The final advantage is: $\hat{A}_{n,i} = \hat{A}_{n,i}^{\text{base}} (1 + b \cdot \hat{A}_{n,i}^{\text{fut}})$, where b controls the contribution of the future-aware term. Finally, APPO optimizes:

$$J(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{X} \\ \mathcal{H} \sim \pi_{\text{old}}(\cdot \mid x)}} \left[\frac{1}{M} \sum_{n=1}^N \frac{1}{|\mathcal{H}_n|} \sum_{i=1}^{|\mathcal{H}_n|} \min(\rho_{n,i}(\theta) \hat{A}_{n,i}, \text{clip}_\epsilon(\rho_{n,i}(\theta)) \hat{A}_{n,i}) - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \right] \quad (8)$$

where clip_ϵ clips the term into $[1 - \epsilon, 1 + \epsilon]$. π_{ref} and π_{old} denote the reference and behavior policies, respectively, and β controls the KL regularization strength. Branches sampled from π_θ are not directly optimized; they provide auxiliary procedural signals for advantage estimation.

3.4 Theoretical Foundation of APPO

We provide a theoretical foundation showing that branching at decision points reduces gradient variance and that the future-aware advantage design admits a policy improvement bound. First, APPO motivates high-impact decision points via BS, leading to the following variance reduction property:

Theorem 3.1 (Variance Reduction) *Let g_{APPO} denote the gradient estimator guided by BS at decision points, and let g_{base} denote the estimator using random branching under the same computational budget. Suppose the conditional reward variance of a decision point D_i is monotone in its branching score BS_i , i.e., $\text{Var}(R \mid D_i) = f(\text{BS}_i)$ with $\nabla f(\cdot) \geq 0$; Then, with $\sigma_i^2 := \text{Var}(R \mid D_i) = f(\text{BS}_i)$ and branches are allocated as $n_i^{\text{APPO}} = M \cdot \sigma_i / \sum_{j=1}^K \sigma_j$. APPO can allocate more samples to high-variance decision points, and the resulting estimator satisfies $\text{Var}(g_{\text{APPO}}) \leq \text{Var}(g_{\text{base}}) - \Delta_\Omega(\text{BS})$, where $\Delta_\Omega(\text{BS}) \geq 0$ denotes the variance reduction induced by branching score.*

We defer the proof to Appendix A.1. Beyond variance reduction, we further introduce Theorem 3.2, which shows that APPO’s advantage design (Eq. 7) admits a policy improvement bound under procedural branching:

Table 1: Performance comparison between APPO and others on 10 challenging Deep Reasoning datasets. Notably the best and suboptimal results are in **bold** and underlined respectively.

Method	Mathematical Reasoning					Knowledge-Intensive Reasoning					Avg.	
	AIME24	AIME25	MATH500	GSM8K	MATH	WebWalker	HotpotQA	2Wiki	Musique	Bamboogle		
Backbone: Llama3.1-8B-Instruct												
Training-Free Method												
Zero-shot	3.3	0.0	43.3	81.4	60.6	3.0	24.3	24.6	10.4	40.0	28.8	
TIR Prompting	3.3	3.3	39.4	73.8	58.2	15.0	48.5	47.5	15.5	58.4	36.3	
Classic RL Method												
GRPO	13.3	<u>13.3</u>	62.4	87.4	79.2	26.5	57.8	71.8	31.0	68.2	51.1	
Reinforce++	13.3	16.7	61.4	87.0	77.2	27.5	57.1	71.6	29.9	69.1	51.1	
DAPO	16.7	<u>13.3</u>	61.2	87.4	76.4	25.5	56.6	70.3	29.2	67.3	50.4	
GPPO	16.7	6.7	61.8	86.6	79.4	27.5	<u>61.8</u>	72.8	29.8	71.9	51.5	
CISPO	13.3	6.7	62.2	87.0	78.2	26.0	57.3	<u>75.6</u>	32.2	71.8	51.0	
Agentic RL Method												
GIGPO	20.0	<u>13.3</u>	62.4	87.4	77.2	<u>31.5</u>	<u>61.8</u>	74.6	31.8	72.1	53.2 Δ_{base}	
ARPO	<u>23.3</u>	16.7	<u>64.6</u>	<u>88.0</u>	<u>80.2</u>	30.5	65.4	75.5	<u>34.8</u>	<u>72.8</u>	<u>55.3</u> +3.9%	
APPO (Ours)	30.0	16.7	69.4	88.6	81.4	32.0	65.4	79.8	35.2	76.8	57.4 +7.9%	
Backbone: Qwen2.5-7B-Instruct												
Training-Free Method												
Zero-shot	10.0	10.0	70.6	90.2	82.0	2.0	12.2	12.6	6.6	24.0	32.0	
TIR Prompting	6.7	10.0	68.2	64.6	78.2	15.5	14.8	18.3	9.5	23.6	30.9	
Classic RL Method												
GRPO	23.3	<u>26.7</u>	78.0	92.8	87.8	22.0	59.0	<u>76.1</u>	30.6	68.4	56.5	
Reinforce++	26.7	23.3	78.0	<u>92.2</u>	<u>88.8</u>	26.0	55.1	<u>68.9</u>	25.2	64.9	54.9	
DAPO	20.0	23.3	<u>80.4</u>	91.0	<u>88.8</u>	24.0	57.7	68.4	28.6	65.5	54.8	
GPPO	26.7	23.3	<u>80.4</u>	91.0	<u>88.8</u>	<u>31.0</u>	<u>60.7</u>	74.2	31.5	<u>72.4</u>	58.0	
CISPO	26.7	30.0	77.8	91.4	86.2	29.0	59.3	72.1	29.1	70.2	57.2	
Agentic RL Method												
GIGPO	<u>30.0</u>	20.0	78.4	91.6	87.6	30.5	58.1	73.5	<u>31.1</u>	70.1	57.1 Δ_{base}	
ARPO	30.0	30.0	78.8	<u>92.2</u>	<u>88.8</u>	26.0	58.8	<u>76.1</u>	<u>31.1</u>	71.5	<u>58.3</u> +2.1%	
APPO (Ours)	36.7	30.0	81.0	93.2	90.0	33.5	66.5	81.5	31.5	77.6	62.2 +8.9%	

Theorem 3.2 (Policy Improvement Bound) Assume $\text{KL}(\pi_{\text{new}} \parallel \pi_{\text{old}}) \leq \epsilon$ and $1 - \epsilon' \leq \hat{A}^{\text{fut}}(s) \leq 1 + \epsilon'$ for all visited states s . Let $\mathcal{J}(\pi)$ denote the expected return, $\omega(s) = 1 + b \hat{A}^{\text{fut}}(s)$, and q the state distribution induced by APPO’s BS-guided branching mixture. Then

$$\mathcal{J}(\pi_{\text{new}}) - \mathcal{J}(\pi_{\text{old}}) \geq \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho^q, a \sim \pi_{\text{new}}} [\omega(s) A^{\pi_{\text{old}}}(s, a)] - \frac{C \epsilon}{(1 - \gamma)^2}, \quad (9)$$

where C depends on r_{max} , b , and ϵ' .

Theorem 3.2 indicates that APPO provides a valid policy improvement bound under BS-guided branching. Detailed proof is provided in Appendix A. Together, these results support decision points as a principled unit for agent exploration and credit assignment.

4 Experiments

4.1 Experiment Setup

Datasets. Following [8, 10], we employ the following three kinds of benchmarks to comprehensively evaluate the model’s capability in long-term complex tool usage: (1) *Mathematical Reasoning*, including the typical math problems like GSM8K [6] and MATH [21]; and competitive math problem like AIME24 [96], AIME25 [97] and MATH500 [43]; (2) *Knowledge-Intensive Reasoning*, including four multi-hop question-answering problems based on Wikipedia, i.e., HotpotQA [89], 2WikiMultihopQA [22], Musique [73] and Bamboogle [54]. We also include the web multi-hop task WebWalker [81]; (3) *Deep Search*, including the challenging General AI Assistant (GAIA) [51], Humanity’s Last Exam [53], and information retrieval tasks like WebWalker [81] and Xbench [4].

Baselines. Four kinds of baselines are selected for comprehensive comparison: (1) *Vanilla RL Method*, including GRPO [18], Reinforce++ [24], DAPO [92], GPPO [68] and CISPO [3]; (2) *Agentic RL Method*, including GIGPO [14] and ARPO [10]; (3) *LLM Backbones*, where we adopt the instruct version of Llama3.1-8B [11] and Qwen2.5 [26] as the generalized backbones; we also report the results of QwQ [72], DeepSeek-R1 [18], GPT-4o [27], o1-preview [27] and Qwen3-32B [88]; (4) *Search Agents and others*, where we adopt the Search-o1 [37] and Webthinker [39]; For other mechanisms we employ the RAG [35] and ReAct [91] framework.

Table 2: Performance comparison between APPO and others on 4 challenging DeepSearch datasets. Results with gray color are from larger / closed-source models and only for reference. Notably the best and suboptimal results are in **bold** and underlined respectively.

Method	General AI Assistant				WebWalkerQA				Humanity’s Last Exam				Xbench
	Lv.1	Lv.2	Lv.3	Avg.	Easy	Med.	Hard	Avg.	NS	CE	SF	Avg.	Avg.
Qwen3-32B-thinking	26.2	12.1	0.0	14.9	6.9	1.1	2.9	3.1	14.6	9.8	8.4	<u>12.6</u>	14.0
DeepSeek-R1-32B	21.5	13.6	0.0	14.2	7.5	1.4	4.2	3.8	6.6	5.1	6.5	6.4	10.0
QwQ-32B	30.9	6.5	5.2	18.9	7.5	2.1	4.6	4.3	11.5	7.3	5.2	9.6	10.7
GPT-4o	23.1	15.4	8.3	17.5	6.7	6.0	4.2	5.5	2.7	1.2	3.2	2.6	18.0
DeepSeek-R1-671B	40.5	21.2	5.2	25.2	5.0	11.8	11.3	10.0	8.5	8.1	9.3	8.6	32.7
o1-preview	/	/	/	/	11.9	10.4	7.9	9.9	12.9	8.1	6.6	11.1	/
<i>Single-Enhanced Method (Qwen3-8B)</i>													
Vanilla RAG	28.2	15.4	16.7	20.4	8.9	10.7	9.9	10.0	5.1	1.6	12.9	5.8	8.0
Search-o1	35.9	15.4	0.0	21.4	6.7	15.5	9.7	11.5	7.6	2.7	5.3	6.4	10.0
WebThinker	43.6	11.5	0.0	22.3	6.7	13.1	16.9	13.0	7.3	4.0	6.3	6.6	13.0
ReAct	35.9	17.3	<u>8.3</u>	23.3	8.9	16.7	<u>18.3</u>	15.5	4.2	4.0	6.3	4.6	16.0
<i>RL-based Method (Qwen3-8B)</i>													
Qwen3-8B	28.1	15.4	16.7	20.4	0.0	2.4	2.8	2.0	3.9	2.7	8.4	4.6	9.0
+GRPO	48.7	25.0	<u>8.3</u>	32.0	28.9	32.1	28.2	30.0	<u>7.9</u>	4.0	10.5	7.8	20.0
+ARPO	<u>53.9</u>	<u>32.7</u>	16.7	<u>38.8</u>	<u>31.1</u>	<u>35.7</u>	28.2	<u>32.0</u>	7.3	<u>6.7</u>	15.8	<u>8.8</u>	<u>25.0</u>
+APPO (Ours)	59.0	36.5	16.7	42.7	37.5	36.4	28.2	33.8	14.7	8.3	<u>13.0</u>	13.4	28.0
<i>Single-Enhanced Method (Qwen3-14B)</i>													
Vanilla RAG	38.5	19.2	8.3	25.2	17.8	13.1	11.3	13.5	5.5	6.3	9.4	6.0	15.0
Search-o1	48.7	23.1	0.0	30.1	11.1	21.4	16.9	17.5	6.4	4.0	10.5	6.8	21.0
WebThinker	48.7	26.9	8.3	33.0	13.3	23.8	18.3	19.5	7.0	4.0	9.5	7.0	23.0
ReAct	48.7	25.0	8.3	32.0	11.1	20.2	12.7	15.5	5.8	5.3	10.5	6.6	20.0
<i>RL-based Method (Qwen3-14B)</i>													
Qwen3-14B	33.3	13.5	0.0	19.4	6.7	2.4	4.2	4.0	5.5	6.7	11.6	6.8	14.0
+GRPO	51.3	34.6	0.0	36.9	35.6	42.9	35.2	38.5	7.9	6.7	12.6	8.6	27.0
+ARPO	<u>56.4</u>	<u>40.4</u>	<u>16.7</u>	<u>43.7</u>	<u>40.0</u>	<u>44.1</u>	<u>36.6</u>	<u>40.5</u>	<u>10.3</u>	<u>10.7</u>	<u>13.7</u>	<u>10.0</u>	<u>32.0</u>
+APPO (Ours)	61.5	42.3	16.7	46.6	43.0	48.6	37.6	43.4	11.5	12.8	14.5	12.3	34.0

Metrics. We employ the F1-score to evaluate the performance of models over four question-answering tasks. For other tasks, we employ LLM-as-a-Judge and deploy a Qwen2.5-72B-Instruct model with vLLM for correctness calculation. We use pass@1 over the LLM-judged results.

Implementation Details. Following [47], we implement the decay rate in procedure advantage as $\gamma = 2^{-\frac{1}{\tau}}$ and keeps $\tau = 32$. The overall methodology is implemented on the VeRL framework [63] where tool-call results are detached from gradient computation. The batch-size is set to 128 and a PPO mini-batchsize of 16, resulting in 8 gradient updates in one step. We also set the coefficient $\beta = 0$ to stabilize training. We train APPO in both reasoning and search tasks following ARPO [10], where models are separately trained for 2 and 5 epochs. The search results are the top-10 snippets retrieved by Bing search engine following [31, 40, 9] and python code is executed in a sandbox environment. To ensure the model possesses basic tool-use capabilities, we directly adopt the SFT pipeline from ARPO [10]. More implementation details are attached to Appendix C.

4.2 Main Results

Results on Mathematical Reasoning and Knowledge-Intensive Reasoning. As shown in Table 1, APPO consistently achieves superior performance across both task groups. We make the following observations: (i) On mathematical reasoning tasks, APPO surpasses all baselines, improving over the previous best agentic RL method by an average of 2.45 points. This suggests that branching at high-impact decision points, rather than fixed turn-level boundaries, enables the model to better explore and reinforce critical deductive paths for complex problem solving. (ii) On knowledge-intensive reasoning tasks, APPO ranks first on nearly all datasets, showing strong performance on multi-hop information synthesis tasks. This indicates that identifying and assigning higher credit to high-impact decision points helps the model strengthen useful procedural reasoning patterns, such as planning, reflection, and verification, leading to consistent gains over both classical RL and prior agentic RL methods that rely on coarser credit assignment.

Results on DeepSearch Tasks. The results in Table 2 demonstrate both the effectiveness and efficiency of APPO in complex deep search scenarios. While even powerful large-scale closed-source models achieve unsatisfactory performance, APPO establishes new state-of-the-art results at both the 8B and 14B model scales, consistently outperforming strong baselines such as ARPO. For instance,

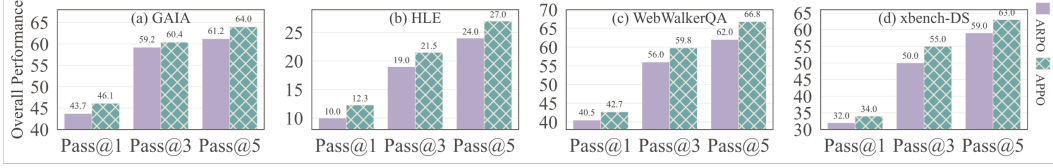


Figure 3: Analysis of Pass@1 to Pass@5 of ARPO and APPO on four datasets respectively.

Table 3: Analysis on branching config ($L = 1$).

Method	Web.	Hotpot.	2Wiki	Musiq.	Bamb.	Avg.
ARPO ($M = 16$)	26.0	58.8	76.1	31.1	71.5	52.7
$M = 4$						
APPO ($N = 2, B = 1$)	25.4	60.4	77.0	30.6	70.2	52.7
APPO ($N = 1, B = 3$)	26.7	63.8	77.6	31.0	70.6	53.9
$M = 8$						
APPO ($N = 2, B = 3$)	28.0	65.0	79.4	30.8	73.3	55.3
$M = 16$						
APPO ($N = 4, B = 3$)	33.5	66.5	81.5	31.5	77.6	58.1
APPO ($N = 8, B = 1$)	31.8	65.8	81.2	31.0	79.5	57.9
APPO ($N = 2, B = 7$)	29.2	65.5	79.0	31.2	75.7	56.1

Table 4: Ablation study on components.

Method	Web.	Hotpot.	2Wiki	Musiq.	Bamb.	Avg.
<i>Llama3.1-8B-Instruct</i>						
APPO	32.0	65.5	79.8	35.2	76.8	57.9
BS→Ent.	31.4	65.4	78.0	34.8	75.5	57.0
w/o $\hat{A}^{\text{fut}}$	29.8	64.8	78.6	35.0	74.5	56.5
w/o Dual-group	30.9	65.0	76.4	35.5	76.2	56.8
<i>Qwen2.5-7B-Instruct</i>						
APPO	33.5	66.5	81.5	31.5	77.6	58.1
BS→Ent.	29.4	64.8	80.5	31.0	75.7	56.3
w/o $\hat{A}^{\text{fut}}$	30.3	60.0	79.9	30.6	72.6	54.7
w/o Dual-group	33.0	63.8	75.8	30.6	76.6	56.0

on the GAIA benchmark, APPO achieves scores of 42.7 and 46.6 with Qwen3-8B and Qwen3-14B, respectively, while also improving performance on HLE. These gains are particularly notable given that deep search tasks involve long horizons and intricate tool-use patterns. The advantage of APPO stems from its decision-point-based branching mechanism: by focusing on high-impact positions that induce divergence in reasoning trajectories, APPO avoids excessive branching on consecutive high-entropy but low-impact tokens, while encouraging more diverse and effective search paths. Consequently, by prioritizing structurally meaningful decision points, APPO enables more targeted exploration and improves the reliability of result-oriented agentic reasoning.

4.3 Scaling Analysis of APPO

Pass@K Analysis. Figure 3 shows that the benefit of APPO extends beyond improving the single best trajectory and instead improves the overall distribution of candidate trajectories, as reflected in the Pass@K results. Across all four datasets, the advantage of APPO is consistently preserved and further enlarged as k increases, indicating that the method improves not only top-1 correctness but also the diversity of valid reasoning paths in the sampling space. On GAIA, Qwen3-14B improves from 43.7 to 46.1 at Pass@1, with the gap further expanding at Pass@5 from 61.2 to 64.0. A similar trend appears on WebWalkerQA, where the gain increases from 40.5 to 42.7 at Pass@1 and from 62.0 to 66.8 at Pass@5. These results suggest that APPO promotes exploration over structurally distinct reasoning trajectories rather than local token-level variations, leading to a broader set of high-quality candidate solutions and significant improvement of pass@K metrics.

Branching Configuration Analysis. Table 3 shows that the effectiveness of APPO depends not only on whether branching is introduced, but also on how the rollout budget is allocated between the number of initial trees N and the number of selected tokens B . Under the same total budget, balanced configurations consistently outperform both extremes. For example, when $M = 16$, APPO with $(N = 4, B = 3)$ achieves the best average score of 58.1, exceeding both $(N = 8, B = 1)$ with 57.9 and $(N = 2, B = 7)$ with 56.1. Increasing N improves the diversity of initial trajectories but leaves less budget for expanding high-impact decision points. In contrast, increasing B enables deeper exploration around these decision points but concentrates the budget on fewer initial paths, reducing global coverage. APPO performs best in the middle regime because decision-point-guided branching is most effective when the model first explores diverse root trajectories and then expands informative internal decisions.

Ablation on Components. Table 4 verifies that all components of APPO contribute to the final gains, with complementary rather than redundant effects. Specifically, replacing the Branching Score (BS) with entropy leads to consistent performance drops of 1.7 and 0.9 points on the two backbones. This suggests that pure entropy fails to prioritize high-impact decision points that are likely to alter downstream reasoning paths. Disabling dual-group advantage estimation also yields a clear degradation, since initial rollouts and branches are generated from different policy distributions and should be compared within their respective groups. Finally, removing $\hat{A}^{\text{fut}}$ causes an even larger drop, especially on Qwen2.5-7B, where the average score decreases from 58.1 to 54.7. Overall, BS

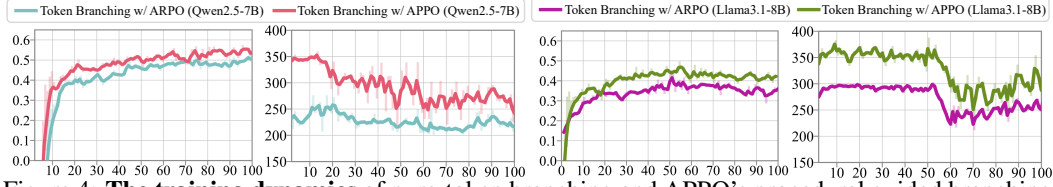


Figure 4: The training dynamics of pure-token branching and APPO’s procedural guided branching.

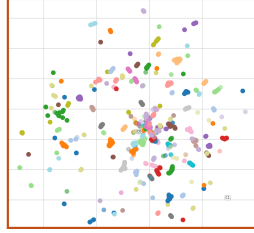
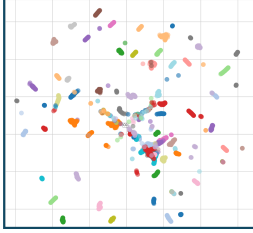


Figure 5: The visualization of the branch distributions from ARPO (left) and APPO (right).



Figure 6: The word cloud of high-entropy tokens and those selected by our BS metric.

improves where the model explores, while dual-group advantage estimation and $\hat{A}^{\text{fut}}$ ensure that branches are compared under appropriate distributions and assigned more fine-grained credit.

4.4 Qualitative Analysis of APPO

Training Dynamics. Figure 4 compares the training dynamics of APPO and ARPO. APPO reaches a higher final reward and follows a more stable improvement trajectory throughout training. This advantage appears early and becomes more pronounced as optimization proceeds, especially on Qwen2.5-7B. These results suggest that APPO does not merely improve local exploration, but allocates exploration more effectively during training. Specifically, APPO branches around high-impact decision points that reflect meaningful differences in reasoning strategy, leading to larger and smoother reward improvements. APPO maintains this advantage on both backbones, indicating that the gains mainly come from the algorithmic design rather than model-specific behavior.

Diversity Analysis. Figure 5 presents the DBSCAN [12] clustering results of rollouts sampled by ARPO and APPO, leveraging UMAP [49]. APPO produces more compact and better-separated clusters than ARPO, whose branches are more diffuse and less structured. This suggests that APPO improves diversity at the level of reasoning strategy. Branches around similar high-impact decision points remain semantically coherent, while branches from different decision points show larger semantic gaps. These results indicate that APPO gains from producing branches that are more structurally distinct and therefore more informative for credit assignment.

Interpretation of the BS metric. We visualize the tokens selected by the pure high-entropy and by our BS metric (dynamically tracked in the experiment and counted finally) in Figure 6. While high-entropy tokens do include salient reasoning keywords such as “verify”, “sum”, and “break”, they also contain many rare nouns like “march”, “november”. We attribute these cases to *long-tailed effects* of the vocabulary, where uncertainty reflects absolute token rarity rather than reasoning difficulty, and optimizing such tokens is unlikely to produce transferable reasoning gains. In contrast, our BS metric filters out these cases by emphasizing tokens with larger downstream distributional shifts between the current and old policies. As a result, it is more likely to select tokens that actually redirect the reasoning trajectory and steer the success and failure of the continuations.

5 Conclusion

We proposed **APPO**, an agentic RL algorithm that shifts branching and credit assignment from coarse tool- or workflow-level units to fine-grained decision points in the generated sequence. APPO uses a Branching Score to select high-value branching locations and introduces an extra future-aware advantage scaling for more targeted credit assignment. Experiments on 13 benchmarks show that APPO consistently outperforms strong baselines while keeping efficient tool-calls. Our findings are also generic and enlightening, suggesting that modeling procedural decisions offers a practical direction for improving exploration and credit assignment in agentic RL.

References

- [1] A. Agarwal, N. Jiang, S. M. Kakade, and W. Sun. Reinforcement learning: Theory and algorithms. *CS Dept., UW Seattle, Seattle, WA, USA, Tech. Rep.*, 32:96, 2019.
- [2] S. Cao, D. Li, F. Zhao, S. Yuan, S. R. Hegde, C. Chen, C. Ruan, T. Griggs, S. Liu, E. Tang, et al. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- [3] A. Chen, A. Li, B. Gong, B. Jiang, B. Fei, B. Yang, B. Shan, C. Yu, C. Wang, C. Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- [4] K. Chen, Y. Ren, Y. Liu, X. Hu, H. Tian, T. Xie, F. Liu, H. Zhang, H. Liu, Y. Gong, et al. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*, 2025.
- [5] X. Chu, H. Huang, X. Zhang, F. Wei, and Y. Wang. GPG: A simple and strong reinforcement learning baseline for model reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [7] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [8] G. Dong, L. Bao, Z. Wang, K. Zhao, X. Li, J. Jin, J. Yang, H. Mao, F. Zhang, K. Gai, et al. Agentic entropy-balanced policy optimization. *arXiv preprint arXiv:2510.14545*, 2025.
- [9] G. Dong, Y. Chen, X. Li, J. Jin, H. Qian, Y. Zhu, H. Mao, G. Zhou, Z. Dou, and J.-R. Wen. Tool-star: Empowering llm-brained multi-tool reasoner via reinforcement learning. *arXiv preprint arXiv:2505.16410*, 2025.
- [10] G. Dong, H. Mao, K. Ma, L. Bao, Y. Chen, Z. Wang, Z. Chen, J. Du, H. Wang, F. Zhang, et al. Agentic reinforced policy optimization. *arXiv preprint arXiv:2507.19849*, 2025.
- [11] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024.
- [12] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pages 226–231, 1996.
- [13] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.
- [14] L. Feng, Z. Xue, T. Liu, and B. An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- [15] X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, and J. Wang. Alphazero-like tree-search can guide large language model decoding and training. *arXiv preprint arXiv:2309.17179*, 2023.
- [16] J. Gao, W. Fu, M. Xie, S. Xu, C. He, Z. Mei, B. Zhu, and Y. Wu. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*, 2025.
- [17] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.
- [18] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [19] Y. Guo, L. Xu, J. Liu, D. Ye, and S. Qiu. Segment policy optimization: Effective segment-level credit assignment in rl for large language models. *arXiv preprint arXiv:2505.23564*, 2025.
- [20] D. Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021.
- [21] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [22] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, 2020.

- [23] Z. Hou, Z. Hu, Y. Li, R. Lu, J. Tang, and Y. Dong. Treerl: Llm reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12355–12369, 2025.
- [24] J. Hu, J. K. Liu, H. Xu, and W. Shen. Reinforce++: Stabilizing critic-free policy optimization with global advantage normalization. *arXiv preprint arXiv:2501.03262*, 2025.
- [25] K. Huang, H. Meng, J. Wu, J. Lu, C. Ma, Z. Chen, X. Wang, B. Ding, J. Wu, X. Wang, et al. On the direction of rlvr updates for llm reasoning: Identification and exploitation. *arXiv preprint arXiv:2603.22117*, 2026.
- [26] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- [27] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [28] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [29] Y. Ji, Z. Ma, Y. Wang, G. Chen, X. Chu, and L. Wu. Tree search for llm agent reinforcement learning. *arXiv preprint arXiv:2509.21240*, 2025.
- [30] J. Jin, X. Li, G. Dong, Y. Zhang, Y. Zhu, Y. Zhao, H. Qian, and Z. Dou. Hira: A hierarchical reasoning framework for decoupled planning and execution in deep search. *arXiv preprint arXiv:2507.02652*, 2025.
- [31] J. Jin, Y. Zhu, Z. Dou, G. Dong, X. Yang, C. Zhang, T. Zhao, Z. Yang, and J.-R. Wen. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 737–740, 2025.
- [32] T. Kaufmann, P. Weng, V. Bengs, and E. Hüllermeier. A survey of reinforcement learning from human feedback. *arXiv preprint arXiv:2312.14925*, 2023.
- [33] X. Lai, Z. Tian, Y. Chen, S. Yang, X. Peng, and J. Jia. Step-dpo: Step-wise preference optimization for long-chain reasoning of llms. *arXiv preprint arXiv:2406.18629*, 2024.
- [34] H. Lee, S. Phatale, H. Mansoor, K. R. Lu, T. Mesnard, J. Ferret, C. Bishop, E. Hall, V. Carbune, and A. Rastogi. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. 2023.
- [35] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [36] P. Li, Z. Gao, B. Zhang, Y. Mi, X. Ma, C. Shi, T. Yuan, Y. Wu, Y. Jia, S.-C. Zhu, et al. Iterative tool usage exploration for multimodal agents via step-wise preference tuning. *arXiv preprint arXiv:2504.21561*, 2025.
- [37] X. Li, G. Dong, J. Jin, Y. Zhang, Y. Zhou, Y. Zhu, P. Zhang, and Z. Dou. Search-o1: Agentic search-enhanced large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5420–5438, 2025.
- [38] X. Li, W. Jiao, J. Jin, G. Dong, J. Jin, Y. Wang, H. Wang, Y. Zhu, J.-R. Wen, Y. Lu, et al. Deepagent: A general reasoning agent with scalable toolsets. *arXiv preprint arXiv:2510.21618*, 2025.
- [39] X. Li, J. Jin, G. Dong, H. Qian, Y. Wu, J.-R. Wen, Y. Zhu, and Z. Dou. Webthinker: Empowering large reasoning models with deep research capability. *arXiv preprint arXiv:2504.21776*, 2025.
- [40] X. Li, J. Jin, Y. Zhou, Y. Wu, Z. Li, Y. Qi, and Z. Dou. Retrollm: Empowering large language models to retrieve fine-grained evidence within generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16754–16779, 2025.
- [41] X. Li, H. Zou, and P. Liu. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025.
- [42] Z.-Z. Li, D. Zhang, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, P.-J. Wang, X. Chen, et al. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025.
- [43] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. In *The twelfth international conference on learning representations*, 2023.

- [44] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [45] K. Lu, C. Chen, X. Wang, B. Cui, Y. Liu, and W. Zhang. Pilotrl: Training language model agents via global planning-guided progressive reinforcement learning. *arXiv preprint arXiv:2508.00344*, 2025.
- [46] Z. Lu, Z. Yao, J. Wu, C. Han, Q. Gu, X. Cai, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- [47] C. Ma, S. Yang, K. Huang, J. Lu, H. Meng, S. Wang, B. Ding, S. Vosoughi, G. Wang, and J. Zhou. Fipo: Eliciting deep reasoning with future-kl influenced policy optimization. *arXiv preprint arXiv:2603.19835*, 2026.
- [48] Z. Ma, S. Yang, Y. Ji, X. Wang, Y. Wang, Y. Hu, T. Huang, and X. Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026.
- [49] L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [50] H. Meng, K. Huang, S. Wei, C. Ma, S. Yang, X. Wang, G. Wang, B. Ding, and J. Zhou. Sparse but critical: A token-level analysis of distributional shifts in rlvr fine-tuning of llms. *arXiv preprint arXiv:2603.22446*, 2026.
- [51] G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.
- [52] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. B. Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332, 2025.
- [53] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- [54] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.
- [55] C. Qian, E. C. Acikgoz, Q. He, H. Wang, X. Chen, D. Hakkani-Tür, G. Tur, and H. Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- [56] C. Qian, E. C. Acikgoz, H. Wang, X. Chen, A. Sil, D. Hakkani-Tur, G. Tur, and H. Ji. Smart: Self-aware agent for tool overuse mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4604–4621, 2025.
- [57] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [58] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [59] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
- [60] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [61] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [62] L. Shen, Y. Zhang, C. K. Ling, X. Zhao, and T.-S. Chua. Carl: Critical action focused reinforcement learning for multi-step agent. *arXiv preprint arXiv:2512.04949*, 2025.
- [63] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

- [64] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [65] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.
- [66] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [67] H. Su, S. Diao, X. Lu, M. Liu, J. Xu, X. Dong, Y. Fu, P. Belcak, H. Ye, H. Yin, et al. Toolorchestra: Elevating intelligence via efficient model and tool orchestration. *arXiv preprint arXiv:2511.21689*, 2025.
- [68] Z. Su, L. Pan, X. Bai, D. Liu, G. Dong, J. Huang, M. Lv, W. Hu, F. Zhang, K. Gai, et al. Klear-reasoner: Advancing reasoning capability via gradient-preserving clipping policy optimization. *arXiv preprint arXiv:2508.07629*, 2025.
- [69] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [70] K. Team, A. Du, B. Gao, B. Xing, C. Jiang, C. Chen, C. Li, C. Xiao, C. Du, C. Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [71] M. L. Team, B. Li, B. Lei, B. Wang, B. Rong, C. Wang, C. Zhang, C. Gao, C. Zhang, C. Sun, et al. Longcat-flash technical report. *arXiv preprint arXiv:2509.01322*, 2025.
- [72] Q. Team. Qwq: Reflect deeply on the boundaries of the unknown, 2024.
- [73] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [74] H. Wang, C. Qian, M. Li, J. Qiu, B. Xue, M. Wang, H. Ji, and K.-F. Wong. Toward a theory of agents as tool-use decision-makers. *arXiv preprint arXiv:2506.00886*, 2025.
- [75] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 2609–2634, 2023.
- [76] P. Wang, S. Xu, J. Li, Y. Luo, D. Li, J. Hao, and M. Zhang. **Re**²: Unlocking llm reasoning via reinforcement learning with re-solving. *arXiv preprint arXiv:2603.07197*, 2026.
- [77] S. Wang, L. Yu, C. Gao, C. Zheng, S. Liu, R. Lu, K. Dang, X. Chen, J. Yang, Z. Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025.
- [78] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [79] S. Welleck, A. Bertsch, M. Finlayson, H. Schoelkopf, A. Xie, G. Neubig, I. Kulikov, and Z. Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models. *arXiv preprint arXiv:2406.16838*, 2024.
- [80] D. Wu, D. S. Sachan, W.-t. Yih, and M. Chen. Procedural knowledge at scale improves reasoning. *arXiv preprint arXiv:2604.01348*, 2026.
- [81] J. Wu, W. Yin, Y. Jiang, Z. Wang, Z. Xi, R. Fang, L. Zhang, Y. He, D. Zhou, P. Xie, et al. Webwalker: Benchmarking llms in web traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10290–10305, 2025.
- [82] J. Wu, J. Zhu, Y. Liu, M. Xu, and Y. Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503, 2025.
- [83] R. Wu, X. Wang, J. Mei, P. Cai, D. Fu, C. Yang, L. Wen, X. Yang, Y. Shen, Y. Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- [84] Y. Wu, Z. Sun, S. Li, S. Welleck, and Y. Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.

- [85] P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [86] Y. Xie, A. Goyal, W. Zheng, M.-Y. Kan, T. P. Lillicrap, K. Kawaguchi, and M. Shieh. Monte carlo tree search boosts reasoning via iterative preference learning. *arXiv preprint arXiv:2405.00451*, 2024.
- [87] R. Xin, C. Xi, J. Yang, F. Chen, H. Wu, X. Xiao, Y. Sun, S. Zheng, and M. Ding. Bfs-prover: Scalable best-first tree search for llm-based automatic theorem proving. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32588–32599, 2025.
- [88] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [89] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
- [90] S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [91] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [92] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [93] Y. Zhai, S. Tao, C. Chen, A. Zou, Z. Chen, Q. Fu, S. Mai, L. Yu, J. Deng, Z. Cao, et al. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.
- [94] G. Zhang, H. Geng, X. Yu, Z. Yin, Z. Zhang, Z. Tan, H. Zhou, Z. Li, X. Xue, Y. Li, et al. The landscape of agentic reinforcement learning for llms: A survey. *arXiv preprint arXiv:2509.02547*, 2025.
- [95] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.
- [96] Y. Zhang and T. Math-AI. American invitational mathematics examination (aime) 2024, 2024.
- [97] Y. Zhang and T. Math-AI. American invitational mathematics examination (aime) 2025, 2025.
- [98] Y. Zhao, W. Huang, S. Wang, R. Zhao, C. Chen, Y. Shu, and C. Qin. Training multi-turn search agent via contrastive dynamic branch sampling. *arXiv preprint arXiv:2602.03719*, 2026.
- [99] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- [100] Y. Zheng, R. Zhang, J. Zhang, Y. Ye, and Z. Luo. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 3: system demonstrations)*, pages 400–410, 2024.
- [101] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Appendix

APPO: Agentic Procedural Policy Optimization

The Appendix of the paper is organized as:

- Appendix A: We give the proof of theorems.
- Appendix B: We introduce datasets and baselines.
- Appendix C: We report full implementation details.
- Appendix D: We report alternative designs of the BS metric.
- Appendix E: We report studies of the branching budget.
- Appendix F: We report the prompts we used.
- Appendix G: We report the limitations.
- Appendix H: We report case studies.
- Appendix I: We report the algorithm of APPO.
- Appendix J: We report the impact statement.
- Appendix K: We report the declaration of LLM usage.

A Mathematical Proof

A.1 Proof of Theorem 3.1

Proof. Let $\mathcal{D} = \{D_1, \dots, D_K\}$ denote the candidate branching locations in one rollout, where each D_i is a fine-grained decision point. Let n_i be the number of branches assigned to D_i , with total budget $\sum_{i=1}^K n_i = M$. The baseline uniform allocation sets $n_i^{\text{uniform}} = M/K$ for all i . By the assumptions, APPO adopts the proportional branch allocation n_i^{APPO} (assigning more branches to higher-BS_{*i*} locations); top-BS selection in practice approximates this rule. For each D_i , let g_i denote the gradient estimator from its branch samples. Assuming i.i.d. branch rewards at D_i , the total estimator variance decomposes as

$$\text{Var}(g) = \sum_{i=1}^K \text{Var}(g_i \mid D_i) = \sum_{i=1}^K \frac{\sigma_i^2}{n_i}, \quad (10)$$

where $\sigma_i^2 := \text{Var}(R \mid D_i)$ as in the theorem. For i.i.d. branch rewards at D_i , $\text{Var}(g_i \mid D_i) = \sigma_i^2/n_i$. By Lagrange multipliers (or Cauchy–Schwarz), the allocation minimizing $\sum_i \sigma_i^2/n_i$ subject to $\sum_i n_i = M$ is $n_i^* = M\sigma_i / \sum_j \sigma_j$. Therefore, for any uniform allocation $n_i^{\text{uniform}} = M/K$,

$$\sum_{i=1}^K \frac{\sigma_i^2}{n_i^{\text{APPO}}} \leq \sum_{i=1}^K \frac{\sigma_i^2}{n_i^{\text{uniform}}} = \frac{K}{M} \sum_{i=1}^K \sigma_i^2,$$

with strict inequality unless all σ_i are equal. Setting $\Delta_{\Omega}(\text{BS}) := \text{Var}(g_{\text{base}}) - \text{Var}(g_{\text{APPO}}) \geq 0$ completes the proof.

A.2 Proof of Theorem 3.2

Proof. Write $\mathcal{J}(\pi) \equiv \eta(\pi)$. By the Performance Difference Lemma,

$$\mathcal{J}(\pi_{\text{new}}) - \mathcal{J}(\pi_{\text{old}}) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho^{\pi_{\text{new}}}, a \sim \pi_{\text{new}}} [A^{\pi_{\text{old}}}(s, a)]. \quad (11)$$

Define the APPO surrogate actually optimized (weighted advantage on the branching mixture distribution ρ^q):

$$L_{\text{APPO}}(\pi_{\text{new}}) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho^q, a \sim \pi_{\text{new}}} [\omega(s) A^{\pi_{\text{old}}}(s, a)], \quad \omega(s) = 1 + b \hat{A}^{\text{fut}}(s). \quad (12)$$

Then:

$$\mathcal{J}(\pi_{\text{new}}) - \mathcal{J}(\pi_{\text{old}}) - L_{\text{APPO}}(\pi_{\text{new}}) = \underbrace{\frac{1}{1-\gamma} \left(\mathbb{E}_{\rho^{\pi_{\text{new}}}, \pi_{\text{new}}} - \mathbb{E}_{\rho^q, \pi_{\text{new}}} \right) [A^{\pi_{\text{old}}}] }_{\text{occupancy mismatch}} + \underbrace{\frac{1}{1-\gamma} \mathbb{E}_{\rho^q, \pi_{\text{new}}} [(1 - \omega(s)) A^{\pi_{\text{old}}}(s, a)] }_{\text{weighting mismatch}}. \quad (13)$$

Since $|A^{\pi_{\text{old}}}(s, a)| \leq r_{\text{max}}/(1 - \gamma)$ and $\|\omega\|_{\infty} \leq 1 + b(1 + \epsilon')$, the simulation lemma (e.g., TRPO analysis [60, 1]) yields

$$\left| \mathbb{E}_{\rho^{\pi_{\text{new}}}, \pi_{\text{new}}} [A^{\pi_{\text{old}}}] - \mathbb{E}_{\rho^q, \pi_{\text{new}}} [A^{\pi_{\text{old}}}] \right| \leq \frac{2\gamma r_{\text{max}}}{(1 - \gamma)^2} \mathbb{E}_{s \sim \rho^{\pi_{\text{new}}}} \text{TV}(\pi_{\text{new}}(\cdot|s), \pi_{\text{old}}(\cdot|s)). \quad (14)$$

Under $\text{KL}(\pi_{\text{new}} \parallel \pi_{\text{old}}) \leq \epsilon$, Pinsker’s inequality bounds the TV term by $O(\sqrt{\epsilon})$; absorbing policy and occupancy errors into a single constant C gives the following equation:

$$\mathcal{J}(\pi_{\text{new}}) - \mathcal{J}(\pi_{\text{old}}) \geq L_{\text{APPO}}(\pi_{\text{new}}) - \frac{C\epsilon}{(1 - \gamma)^2}. \quad (15)$$

□

B Datasets and Baselines.

The datasets employed in our experiments are introduced as the following:

AIME24. AIME24 comprises the 30 problems from the 2024 American Invitational Mathematics Examination (AIME), split across two competition papers. Each problem demands an integer answer between 0 and 999, yet the reasoning paths required are far from trivial—contestants must navigate number theory, combinatorics, geometry, and algebra. Its small size makes it sensitive to variance, so performance is typically reported as average accuracy over multiple runs.

AIME25. AIME25 follows the same format as AIME24, providing another 30 competition-level problems released in February 2025. Because the problems post-date most training cutoffs, AIME25 serves as a relatively contamination-resistant probe of genuine mathematical reasoning. It has quickly become a standard checkpoint for evaluating frontier models on olympiad-style problem solving.

MATH500. MATH500 [43] is a 500-problem subset drawn from the broader MATH to span all seven subject categories and five difficulty levels of the original collection. Problems range from introductory algebra to competition-level precalculus, making the subset a compact yet representative testbed for mathematical reasoning. It is widely used as an in-distribution evaluation complement to harder olympiad benchmarks.

MATH. MATH [21] contains 12,500 problems sourced from high school mathematics competitions, covering algebra, counting and probability, geometry, number theory, and precalculus, among others. Each problem is accompanied by a step-by-step solution, enabling process-level as well as answer-level evaluation. The dataset is notable for its difficulty gradient: even the easiest problems require multi-step symbolic reasoning, while the hardest rival olympiad-level challenges.

GSM8K. GSM8K is released by [6] at OpenAI and consists of 8,500 linguistically diverse word problems pitched at the elementary school level. Despite their apparent simplicity, the problems require chains of two to eight arithmetic steps, making them a reliable probe of basic multi-step reasoning. The dataset is split into 7,500 training and 1,000 test examples and remains one of the most widely cited benchmarks for evaluating arithmetic reasoning in language models.

HotpotQA. HotpotQA [89] contains roughly 113,000 Wikipedia-based question–answer pairs that are explicitly designed to require reasoning over two supporting documents. A key design feature is the inclusion of sentence-level supporting-fact annotations, which allow evaluation of both answer correctness and the quality of the reasoning chains.

2WikiMultihopQA. 2WikiMultihopQA [22] constructs multi-hop questions by combining information from two Wikipedia articles, using structured Wikidata triples to guarantee that each question genuinely requires cross-document reasoning rather than being solvable from a single passage. The dataset provides explicit evidence chains alongside each question, enabling fine-grained evaluation of intermediate reasoning steps.

MuSiQue. MuSiQue [73] takes a bottom-up approach to constructing multi-hop questions: single-hop questions from existing datasets are composed into 2–4 hop chains via directed acyclic graphs, ensuring that each hop is genuinely necessary and that shortcuts are systematically eliminated. The dataset contains approximately 25,000 questions and is considered harder to game than earlier multi-hop benchmarks, where models could often answer correctly by attending to a single passage.

Bamboogle. Bamboogle [54] is a small, hand-crafted collection of 125 two-hop factual questions, assembled specifically by filtering out any question that Google’s search engine answers correctly. This adversarial construction criterion means the questions tend to require non-obvious compositional reasoning that surface-level retrieval struggles with. Despite its modest size, Bamboogle is frequently used as a stress test for retrieval-augmented generation systems.

GAIA. GAIA [51] is a benchmark for General AI Assistants comprising 466 real-world questions across three difficulty levels. What distinguishes GAIA is its emphasis on practical, tool-assisted problem solving: questions may require web browsing, file parsing, multi-modal understanding, and multi-step planning in combination. Humans solve the benchmark with high accuracy, yet agents still struggle significantly, positioning GAIA as a meaningful frontier for agentic evaluation.

HLE. Humanity’s Last Exam (HLE) [53] consists of 2,500 questions assembled by over 1,000 subject-matter experts to represent the frontier of human knowledge. Questions span mathematics, the natural sciences, humanities, and professional domains, with roughly 10% requiring image comprehension. The benchmark was explicitly designed to resist saturation: at release, top models scored below 10%, making it one of the most demanding closed-ended evaluations currently available.

WebWalker. WebWalker [81] benchmarks LLMs on web traversal. The accompanying dataset, WebWalkerQA, contains 680 questions derived from real websites, where answering correctly requires an agent to plan a sequence of click actions across multiple pages. The benchmark highlights a gap between static retrieval and dynamic, navigation-based information seeking.

XBench. XBench [4] is a profession-aligned, dynamic evaluation suite introduced by Sequoia Capital to measure AI agent productivity in real-world occupational contexts. Rather than testing isolated capabilities, it presents tasks drawn from domains such as marketing, software engineering, and legal work, scored against expert-produced reference outputs. Its evergreen design is intended to resist benchmark saturation and track whether agent performance translates into genuine workplace utility.

The baselines compared in our experiments are shown in the following:

GRPO. GRPO [18] is critic-free, which groups multiple responses per prompt and utilizing relative reward comparisons within these groups, it optimizes policies based on intra-group performance rather than absolute scalar values, thereby improving sample efficiency.

Reinforce++. REINFORCE++ [24] is an enhanced variant of the classical REINFORCE algorithm that incorporates key optimization techniques from PPO—including token-level KL penalties and reward normalization—while eliminating the need for a critic model. It achieves three primary objectives: simplicity, enhanced training stability, and reduced computational overhead, making it a practical and efficient baseline for RLHF-based LLM alignment.

DAPO. DAPO [92] is an open-source, industrial-scale RL system for LLM post-training proposed by ByteDance. It introduces four key techniques—Clip-Higher, dynamic sampling, token-level policy gradient loss, and overlong reward shaping—to address entropy collapse and training instability commonly observed in GRPO-based long chain-of-thought training.

GPPO. GPPO [68] addresses the gradient vanishing problem caused by hard token-level clipping in standard PPO-style training. By gently reintroducing bounded gradients from clipped tokens, GPPO enables finer-grained exploration control and more stable policy updates, serving as the foundation for the entropy-regularized extension CE-GPPO.

CISPO. CISPO [3] is a reinforcement learning algorithm proposed in the MiniMax-M1 project that clips importance-sampling weights at the sequence level rather than applying per-token update clipping. This design allows all tokens—including low-probability ones—to contribute to gradient updates, reducing variance and improving training stability in off-policy LLM fine-tuning scenarios.

GIGPO. GiGPO [14] is a critic-free RL algorithm designed for training long-horizon LLM agents, proposed by researchers at Nanyang Technological University. It introduces a two-level grouping

structure that combines episode-level advantage estimation with step-level credit assignment, enabling fine-grained attribution of rewards to individual actions without requiring an additional value network.

ARPO. ARPO [10] is a reinforcement learning framework specifically designed for multi-turn, tool-augmented LLM agents. It addresses the entropy increase observed after tool interactions by introducing entropy-based adaptive sampling and fine-grained advantage attribution at the action level, enabling more effective exploration and stable policy optimization in agentic settings.

Qwen3. Qwen3 [88] is the latest generation of the Qwen large language model family, released in April 2025. It introduces a hybrid thinking mode that allows flexible switching between deliberate chain-of-thought reasoning and fast non-thinking inference, supports 119 languages, and was trained on a corpus exceeding 36 trillion tokens, achieving competitive performance across a broad range of benchmarks.

DeepSeek-R1. DeepSeek-R1 [18] is a large reasoning model developed by DeepSeek that incentivizes complex reasoning capabilities through large-scale reinforcement learning with verifiable rewards, without relying on supervised chain-of-thought data in its initial training stage. The resulting model exhibits emergent reasoning behaviors such as self-reflection and dynamic strategy adjustment, achieving performance competitive with OpenAI’s o1 series on mathematical and scientific benchmarks.

QwQ. QwQ [72] is an open-source reasoning model series developed by the Qwen team, first released in November 2024, designed to tackle problems requiring deep analytical thinking. The flagship QwQ-32B model demonstrates that a mid-sized model can achieve competitive performance against state-of-the-art reasoning systems such as DeepSeek-R1 and OpenAI o1-mini through extended chain-of-thought reasoning.

GPT-4o. GPT-4o [27] is OpenAI’s flagship multimodal generative model released in May 2024. It achieves significantly lower latency than its predecessors by natively handling cross-modal inputs without modality-specific preprocessing pipelines, establishing a new standard for real-time multimodal AI interaction.

RAG. RAG [35] enhances large language model outputs by dynamically retrieving relevant documents from an external knowledge base at inference time, thereby grounding generation in up-to-date and domain-specific information. By decoupling parametric knowledge stored in model weights from non-parametric knowledge retrieved on demand, RAG substantially reduces hallucination and improves factual accuracy without requiring full model retraining.

Search-o1. Search-o1 [37] augments large reasoning models with an agentic search workflow, enabling them to dynamically retrieve external knowledge when they encounter uncertain or knowledge-intensive steps during long chain-of-thought reasoning. By seamlessly interleaving retrieval with reasoning rather than treating search as a preprocessing step, Search-o1 improves both the reliability and factual grounding of complex multi-step inference.

WebThinker. WebThinker [39] is a deep research agent that empowers large reasoning models to autonomously search the web, navigate across multiple web pages, and synthesize retrieved information into structured research reports. It introduces a Deep Web Explorer module that tightly integrates web interaction with chain-of-thought reasoning, enabling end-to-end autonomous research on complex, knowledge-intensive tasks.

ReAct. ReAct [91] is a prompting paradigm that interleaves verbal reasoning traces with executable actions in large language models, enabling them to dynamically plan, retrieve information, and adjust their behavior based on environmental feedback. By synergizing chain-of-thought reasoning with tool use, ReAct substantially improves interpretability and task performance on knowledge-intensive decision-making benchmarks compared to reasoning-only or acting-only baselines.

C Implementation Details

Our APPO follows a training pipeline largely consistent with ARPO. Specifically, we first train the backbone model to obtain fundamental tool-use capabilities using ToolStar’s 54K SFT dataset together with an additional 0.8K samples drawn from the STILL dataset. For SFT training, we employ LLAMA-Factory [100] with a learning rate of $7e-6$, DeepSpeed ZeRO-3 [59], and Flash-

Consider a multi-layer rollout tree, i.e., the case where $L > 1$, we have $M = B \cdot (N + 1)^L$. The results of our sensitivity analysis are presented in Table 5.

We observed that: (i) When $N = 1$, the overall performance of the model is relatively poor, showing even a slight decline compared to the case in the main paper where $L = 1$ and M is even smaller. For example, with $(N = 1, B = 2, L = 3, M = 16)$, the overall performance is approximately 54.1%, which is about 1.1 points lower than the case $(N = 2, B = 3, L = 1, M = 8)$ reported in Table 3. We attribute this to the insufficient diversity of the initial rollout, which causes the model to be significantly affected by randomness, leading to performance fluctuations. (ii) When the initial rollout is kept the same, varying B or L does not have a noticeable impact on performance. We believe this is because the two settings largely share the prefix information of the rollouts, resulting in substantial overlap in the sampling distribution. These results once again confirm that our choice in the main text is an optimal trade-off.

F Prompts

Following [10], The prompt used in APPO is listed in the following:

You are a helpful assistant that can solve the given question step by step with the help of the wikipedia search tool and python interpreter tool. Given a question, you need to first think about the reasoning process in the mind and then provide the answer. During thinking, you can invoke the wikipedia search tool to search and python interpreter tool to calculate the math problem for fact information about specific topics if needed. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags respectively, and the search query and result are enclosed within <search> </search> and <result> </result> tags respectively. For example, <think> This is the reasoning process. </think> <search> search query here </search> <result> search result here </result> <think> This is the reasoning process. </think> <python> python code here </python> <result> python interpreter result here </result> <think> This is the reasoning process. </think> <answer> The final answer is answerhere </answer>. In the last part of the answer, the final exact answer is enclosed within $\square$ with latex format.

G Limitations

Although the proposed APPO introduces innovations in both how to branch and where to branch, our work still has the following limitations: (i) The splitting point selection method is validated solely through experiments, lacking theoretical guarantees that BS is an optimal branching criterion. However, fully quantifying the actual branching value of a specific point requires a more systematic framework design and an exploration of the intrinsic properties of LLMs. (ii) Following prior work, tools employed by our APPO are currently restricted to Search and Python, and have not been extended to other application-oriented tools. Despite these factors, APPO demonstrates sufficient performance advantages across the broadest possible range of experimental settings.

H Case Study

We provide two kinds of cases in this section: (i) the branching stage cases, where the initial rollout is wrong but turns correct by our branching selections; (ii) the inference cases of ARPO and APPO, where ARPO fails but solvable to our APPO.

I Algorithm

The algorithm of APPO is shown in Algorithm 1.

J Impact Statement

This paper introduces APPO, a novel Agent RL method designed to push the boundaries of where to branch and how to branch. The proposed APPO holds broad societal value, playing a significant

Algorithm 1: APPO Training Pipeline

Input : policy π_θ , reference policy π_{ref} , toolset T , training set $\mathcal{D}$, rollout budget M , number of initial rollouts N , number of selected branching points B , PPO epochs K_{ppo} , clipping thresholds ϵ, ϵ' , procedural weight b , decay factor γ

for each training step do

- sample input $x \sim \mathcal{D}$
- set behavior policy $\pi_{\text{old}} \leftarrow \pi_\theta$
- $\mathcal{T}_{\text{init}} \leftarrow \emptyset$
- // Initialization: generate initial rollouts**
- for $n = 1$ to N do**
 - generate a full rollout $\mathcal{H}_n \sim \pi_{\text{old}}(\cdot \mid x; T)$ through agent-environment interaction
 - add $\mathcal{H}_n$ to $\mathcal{T}_{\text{init}}$
- // Mini-batch procedural branching and optimization**
- for $e = 1$ to K_{ppo} do**
 - $\mathcal{T}_{\text{branch}} \leftarrow \emptyset$
 - while $|\mathcal{T}_{\text{branch}}| < M - N$ do**
 - uniformly sample one rollout $\mathcal{H}_n$ from the current rollout trees
 - for each valid token position i in $\mathcal{H}_n$ do**
 - compute token entropy $H_{n,i}$ by Eq. (3)
 - compute future value $\Omega_{n,i}$ by Eq. (4)
 - compute Branching Score $\text{BS}_{n,i}$ by Eq. (5)
 - select the top- B tokens according to $\text{BS}_{n,i}$ and denote them as $\mathcal{B}_n$
 - foreach $i \in \mathcal{B}_n$ do**
 - resample a continuation from prefix $\mathcal{H}_{n,<i}$ using the current policy π_θ to obtain a branch $\mathcal{H}_n^{\text{new}}$
 - add $\mathcal{H}_n^{\text{new}}$ to $\mathcal{T}_{\text{branch}}$
 - update the rollout tree with $\mathcal{H}_n^{\text{new}}$
 - if $|\mathcal{T}_{\text{branch}}| = M - N$ then**
 - break**
 - // Dual-group advantage estimation**
 - evaluate reward $R(\mathcal{H})$ for each rollout $\mathcal{H} \in \mathcal{T}_{\text{init}} \cup \mathcal{T}_{\text{branch}}$
 - compute group-relative advantages separately for $\mathcal{T}_{\text{init}}$ and $\mathcal{T}_{\text{branch}}$ by Eq. (6)
 - // Future-aware procedural advantage**
 - for each token position i in each initial rollout $\mathcal{H}_n \in \mathcal{T}_{\text{init}}$ do**
 - compute $\hat{A}_{n,i}^{\text{fut}}$ by Eq. (7)
 - set $\hat{A}_{n,i} \leftarrow \hat{A}_{n,i}^{\text{base}} (1 + b \cdot \hat{A}_{n,i}^{\text{fut}})$
 - // Policy optimization on initial rollouts**
 - optimize Eq. (8) on $\mathcal{T}_{\text{init}}$ with KL regularization to π_{ref}
 - update θ

role in domains such as search systems, AI-assisted healthcare and education. It is particularly worth noting that, as a work aimed at advancing agent capabilities, APPO not only delivers broader efficacy improvements but also catalyzes increased automated behaviors. Like any research in this field, we are fully committed to participating in regulatory compliance and safety governance for agents. Overall, our proposed APPO offers extensive value to both the research and application communities, fostering healthy and sustainable development in the field of agents.

K Declaration of LLM Usage

LLMs are only for polishing the writing of this paper.

Case-1 from MATH

The 7th and 8th grades have enrollments of 520 and 650 respectively. The two grades have a total of 18 representatives on the Student Council. How many representatives should the 8th grade have if there is to be a fair representation of the two grades?

Wrong Rollout

To determine the number of representatives the 8th grade should have, we need to ensure that the number of representatives is proportional to the enrollment of each grade. Here are the steps to solve the problem:

1. Calculate the total enrollment: Total enrollment = $520 + 650 = 1170$
2. Determine the proportion of the 8th grade's enrollment to the total enrollment: Proportion of 8th grade = $650 / 1170$
3. Simplify the proportion: To simplify $650/1170$, we find the greatest common divisor (GCD) of 650 and 1170. The GCD is 50.
 $650 \div 50 / 1170 \div 50 = 13 / 23.4$
4. Express the simplified proportion: $13 / 23.4$
5. Calculate the number of representatives for the 8th grade: Number of representatives = $13 / (13 + 23.4) \times 18$
6. Further calculation: $13 / 36.4 \times 18$
7. Perform the multiplication: $13 / 36.4 \approx 0.35714$
 $0.35714 \times 18 \approx 6.42852$
8. Round to the nearest whole number: Since the number must be a whole number, we round 6.42852 to 6.

Thus, the 8th grade should have 6 representatives.

APPO Branching

To determine the number of representatives the 8th grade should have, we need to ensure that the number of representatives is proportional to the enrollment of each grade. Here are the steps to solve the problem:

1. Calculate the total enrollment: Total enrollment = $520 + 650 = 1170$
2. Determine the proportion of the 8th grade's enrollment to the total enrollment: Proportion of 8th grade = $650 / 1170$
3. Simplify the proportion: To simplify $650/1170$, we find the greatest common divisor (GCD) of 650 and 1170. The GCD is 50.
 $650 \div 50 / 1170 \div 50 = 13 / 23.4$
4. Express the simplified proportion: $13 / 23.4$
5. Calculate the number of representatives for the 8th grade: Number of representatives = $13 / (13 + 23.4) \times 18$
6. Further calculation: $13 / 36.4 \times 18$
7. Perform the multiplication: $13 / 36.4 \approx 0.35714$
 $0.35714 \times 18 \approx 6.42852$
8. Round to the nearest whole number: Since the number must be a whole number, we round 6.42852 to 6.

~~Branching~~

Wait, let me reconsider. The GCD of 650 and 1170 should be recalculated.

$$650 = 2 \times 5^2 \times 13 \quad 1170 = 2 \times 3^2 \times 5 \times 13$$

$$\text{GCD}(650, 1170) = 2 \times 5 \times 13 = 130$$

$$\text{So: } 650 / 1170 = 650 \div 130 / 1170 \div 130 = 5 / 9$$

$$\text{Number of 8th grade representatives} = (5/9) \times 18 = 10$$

Thus, the 8th grade should have 10 representatives.

Case-2 from Musique

What does the acronym of the organization Danish Football Union is part of stand for?

Wrong Rollout

To provide the accurate answer, I need to first identify the organization Danish Football Union is a part of and then find out the acronym associated with that organization.

<search>

search query: What is Danish Football Union organization?

</search>

<result>

Five pages omitted.

</result>

The search result indicates that the Danish Football Union (DBU) is a member of both FIFA and UEFA. Now, I will find the acronym for UEFA to provide the answer.

<search>

search query: What is UEFA acronym?

</search>

<result>

Eight pages omitted.

</result>

Therefore, the acronym of the organization that the Danish Football Union is part of stands for Union of European Football Associations.

APPO Branching

To provide the accurate answer, I need to first identify the organization Danish Football Union is a part of and then find out the acronym associated with that organization.

<search>

search query: What is Danish Football Union organization?

</search>

<result>

Five pages omitted.

</result>

The search result indicates that the Danish Football Union (DBU) is a member of both FIFA and UEFA. Now, I will find the acronym for UEFA to provide the answer.

<search>

search query: What is UEFA acronym?

</search>

<result>

Eight pages omitted.

</result>

-----Branching-----

The search result shows that the Danish Football Union is a member of both FIFA and UEFA. The question asks for "the organization" (singular), and since FIFA is the global governing body of football while UEFA is only the European body, the more fundamental organization is FIFA.

<search>

search query: What is FIFA acronym?

</search>

<result>

Nine pages omitted.

</result>

Therefore, the acronym FIFA stands for International Federation of Association Football.

Case-3 (Tool-Star RL Dataset): when the composer of the film Waqt (1965) was born?

<think>

To find out when the composer of the film Waqt (1965) was born, we need to gather relevant information through Wikipedia searches and identify the necessary details. </think>

<search>search query: Who composed music for Waq 1965 film? </search>

<result>

Page 1: Lyrics and video of songs from Movie / Album : Waqt (1965); Music by: Ravi; Singer (s): Asha Bhosle, Mahendra Kapoor, Manna Dey, Mohammed Rafi; having star cast: Sunil Dutt, Sadhana, Raaj Kumar, ... Page 2: Waqt songs are composed by Ravi Shankar Sharma (Ravi), while Sahir Ludhianvi wrote its lyrics. Check out Waqt songs list with lyrics and music videos below. Page 3: Ravi (music director) — Ravi Shankar Sharma, better known by his first name Ravi, is an Indian music director, who has composed music for several Hindi and Malayalam films. Page 4: These included story writer Akhtar Mirza, who was credited as F.A. Mirza, dialogue writer Akhtar-Ul-Iman, and editor Pran Mehra. Album details and songs for Waqt. Page 5: The film contains the Hindi songs "Hum Jab Simat Ke", sung by Mahendra Kapoor and Asha Bhosle, "Waqt se din aur raat", sung by Mohammed Rafi, "Aage Bhi Jaane Na Tu", sung by Asha Bhosle and ... Page 6: Listen to Waqt (Original Motion Picture Soundtrack) by Ravi, Sahir Ludhianvi & Traditional on Apple Music. 1965. 21 Songs. Duration: 1 hour 35 minutes. </result>

<think>

Having obtained the relevant information, it is clear that Ravi composed music for this film. Now, to ascertain his date of birth, more focused searches would be necessary to obtain accurate information related to him. </think>

<search>

search query: When was Ravi (music director) born? </search>

<result>

Page 1: Ravi was born on 3 March 1926 in Delhi, British India. He was a music artist and composer, known for Gharana (1961), Dilwale Dulhania Le Jayenge (1995) and Khandan (1965). Page 2: Ravi Shankar Sharma (3 March 1926 – 7 March 2012), often referred to mononymously as Ravi, was an Indian music director, who had composed music for several Hindi and Malayalam films. </result>

<think>

Based on the search results found so far, it looks like Ravi (full name Ravi Shankar Sharma) has a birth date which seems to repeat often, but it appears consistent across multiple relevant sources namely March 3, 1926. </think>

<python>

print("Ravi's complete name is Ravi Shankar Sharma. His birthday is on 3rd March 1926.")

</python>

<result>

Ravi's complete name is Ravi Shankar Sharma. His birthday is on 3rd March 1926. </result>

<think>

Upon confirming the date of birth via the reliable resource, Ravi was born on March 3, 1926 which will correctly identify the composer's date of birth without any further steps. </think>

Final confirmation composer_birth_date = "March 3, 1926"

<python>

print(f"The composer of the film 'Waqt (1965)', Ravi was born on composer_birth_date.")

</python>

ground_truth: 3 March 1926 fl_score: 1.0,

answer: March 3, 1926, ground_truth: 3 March 1926

Case-4 (Tool-Star RL Dataset): what award the director of "The River's End (Film)" might have earned?

<think> To find out what award the director of "The River's End (Film)" might have earned, we will start by identifying the director's name from searching on Wikipedia, then proceed further to see if any awards related to him/her can be identified from provided context or additional searches.</think>

<search> search query: Who directed the film 'The River's End'?</search>

<result> Page 1: Warner Bros. Pictures produced River's End, based on the same source in 1930; Michael Curtiz directed, and Charles Bickford and Evelyn Knapp starred (see AFI Catalog of Feature Films, 1921 ... Page 2: Where to watch JustWatch The River's End 1920 Directed by Victor Heerman, Marshall Neilan

six pages omitted.</result>

<think>

It appears there may have slight confusions between different films titled "The River's End". We'll assume that it refers to 'The River's End' directed by Ray Enright which seems consistent across multiple references and is more modern context than others. Now let me try finding director Ray Enright's awards.</think>

<search>

search query: What award did Ray Enright receive as a director?</search> <result> Page 1:

Ray Enright (25 March 1896 – 3 April 1965) was an American film director. He directed 73 films between 1927 and 1953. He was born in Anderson, Indiana and died in Hollywood, California from a heart ... Page 2: In 1926 he started work for Warner Brothers as a cutter and two years later was made a director. The first film he directed was Tracked by the Police (1927), starring Rin Tin Tin. Page 3: Ray Enright was a director and writer who was born in 1896 in United States and died in 1965 known for Montana, The Spoilers, Gung Ho!, South of St. Louis, Kansas Raiders, Coroner Creek, Return of the ... Page 4: Enright directed Humphrey Bogart in 1941's "The Wagons Roll at Night"; the same year, his film "Teddy The Rough Rider" won an Academy Award for Best Short Subject. Page 5: Enright, Ray 1 reference stated in Czech National Authority Database mub20191040662 section Nationale Thesaurus voor Auteursnamen ID 256538646 1 reference stated in Virtual International ... Page 6: Note: Names and film titles in the Academy Awards Database are derived from film credits and other documentation submitted to the Academy at the time of nomination or award. Changes to name ... Page 7: Ray Enright American film director / From Wikipedia, the free encyclopedia Ray Enright (March 25, 1896 – April 3, 1965) was an American film director. He directed 73 films between 1927–53, many of them ... </result>

<think>

From my search results, it mentions that Ray Enright's film "Teddy The Rough Rider" (which appears to be a short subject film, as indicated by context of award categories) won an Academy Award for Best Short Subject in 1941. Hence, the award should be identified accordingly.</think>

<search>

search query: Which award did Ray Enright win for "Teddy the Rough Rider"?</search>

<result>

Page 1: Teddy, the Rough Rider is a 1940 American short drama film directed by Ray Enright. It won an Oscar at the 13th Academy Awards for Best Short Subject (Two-Reel). Page 2: Enright directed Humphrey Bogart in 1941's "The Wagons Roll at Night"; the same year, his film "Teddy The Rough Rider" won an Academy Award for Best Short Subject.

four pages omitted.

</result>

<think>

The information clearly outlines that "Teddy the Rough Rider", directed by Ray Enright, won the Academy Award for Best Short Subject (Two-Reel) at the 13th Academy Awards.</think>

<python> # Confirm the exact award name print("The award for 'Teddy the Rough Rider' is Best Short Subject.") print("Specifically, the category was Best Short Subject (Two-Reel) at the 13th Academy Awards.")

</python>

fl_score: 1.0,

answer: Best Short Subject, ground_truth: Best Short Subject